

Tài liệu này viết cho ai? Người mới clone repo về máy và muốn (1) cài đặt cho chạy được, (2) tạo ra một video thật càng nhanh càng tốt.

Nguồn của tài liệu: đọc trực tiếp source code trong repo này ([Makefile](#), [render_demo.py](#), [lib/](#), [tools/](#), [pipeline_defs/](#), [tests/qa/](#), [.github/workflows/ci.yml](#)). Mỗi mục quan trọng đều kèm con trỏ `file:` để bạn tự tra cứu.

Mức độ kiểm chứng: các lệnh dưới đây lấy nguyên văn từ source. Riêng phần `render` chưa chạy được trong môi trường soạn tài liệu vì sandbox chặn tiến trình `node/npm` — xem mục [12.3](#).

Có **3 đường đi** để tạo video, xếp theo độ khó tăng dần:

Yêu cầu tối thiểu để bắt đầu ngay: Python 3.10+, FFmpeg, Node.js 18+ (Node $\geq$ 22 nếu muốn dùng HyperFrames).

Nếu chỉ muốn thấy một video MP4 xuất hiện càng sớm càng tốt $\rightarrow$ nhảy tới [mục 7](#). Nếu muốn hiểu mình đang cài cái gì $\rightarrow$ đọc [mục 2](#).

Đây là điểm khác biệt lớn nhất so với các "AI video tool" khác. Trong OpenMontage:

Agent đọc pipeline manifest (YAML)	→ biết TIẾN TRÌNH gồm những stage nào
Agent đọc stage director skill (MD)	→ biết CÁCH làm từng stage
Agent gọi tool (Python BaseTool)	→ tay chân: gọi API, render, mix audio
Agent tự review (meta skill)	→ chăm chất lượng theo review_focus
Agent ghi checkpoint (JSON)	→ lưu trạng thái, resumable
Agent trình bày cho người duyệt	→ bạn giữ quyền quyết định

Hệ quả thực tế khi bạn dùng repo:

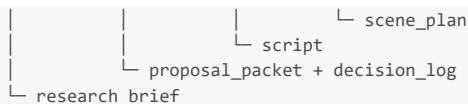
- Muốn đổi hành vi của pipeline → sửa file YAML/Markdown, **không phải sửa logic Python**.
- Muốn biết hệ thống có tool gì → **hỏi registry**, đừng đọc doc cũ (xem [mục 6](#)).
- Chất lượng video phụ thuộc vào việc agent có đọc skill hay không. Nếu bạn tự viết script gọi tool trực tiếp, bạn đang bỏ qua toàn bộ "chất xám" của hệ thống.

Lớp	Nằm ở đâu	Trả lời câu hỏi
1	<code>tools/, pipeline_defs/</code>	"Có những gì?" — tool khả dụng, runtime, chi phí, pipeline
2	<code>skills/</code>	"OpenMontage muốn dùng nó thế nào?" — quy ước, chất lượng, quy trình
3	<code>.agents/skills/</code>	"Công nghệ đó hoạt động ra sao?" — kiến thức vendor/API gốc

Mỗi tool khai báo `agent_skills[]` để trả từ lớp 1 → lớp 3. Trích `PROJECT_CONTEXT.md`:35-41.

Pipeline `animated-explainer` (`pipeline_defs/animated-explainer.yaml`) chạy tuần tự:





Mỗi stage:

1. **Sinh đúng một artifact chuẩn** (cột phải ở sơ đồ trên), validate bằng JSON Schema trong `schemas/artifacts/`.
2. **Ghi checkpoint** `projects/<id>/checkpoint_<stage>.json`.
3. **Có thể là "cổng" (gate)** — stage gated bắt buộc `human_approved=True` mới được ghi `completed`, nếu không `lib/checkpoint.py` ném lỗi GATE VIOLATION (`lib/checkpoint.py:251`).

Các cổng duyệt mặc định của `animated-explainer`: `proposal`, `script`, `scene_plan`, `assets`, `publish` (`human_approval_default: true`). Riêng `research`, `edit`, `compose` chạy tự động.

Ngoài ra còn **một cổng thứ hai ít được biết**: `_enforce_stage_prerequisites` (`lib/checkpoint.py:284-346`). Khi ghi `completed/awaiting_human`, **mọi stage trước đó** trong manifest phải tồn tại, hợp lệ, đúng `project_id/pipeline_type`, có `status == "completed"`, và nếu là stage gated thì phải có `human_approved`. Vi phạm → lỗi `PREREQUISITE VIOLATION`. Cổng này **không được mô tả** trong `AGENT_GUIDE.md` lẫn `PROJECT_CONTEXT.md` — nếu bạn tự viết script ghi checkpoint, đây là lỗi rất dễ gặp.

Giá trị `status` hợp lệ **chỉ có 4**: `completed`, `failed`, `awaiting_human`, `in_progress` (`schemas/checkpoints/checkpoint.schema.json:17`). Không có `pending`.

Điều này nghĩa là gì với bạn? Ở kịch bản 3, agent sẽ **dừng lại và chờ bạn duyệt** ở các stage trên. Đây là thiết kế có chủ đích, không phải lỗi. Đừng tắt nó nếu bạn chưa quen.

2.4 Cây thư mục

```

OpenMontageVideo/
├── AGENT_GUIDE.md           # HỢP ĐỒNG vận hành cho AI agent – đọc trước tiên
├── PROJECT_CONTEXT.md       # Kiến trúc + quy ước
├── README.md                # Giới thiệu, quickstart gốc
├── config.yaml              # Cấu hình toàn cục (llm, budget, output, paths)
├── .env.example             # Mẫu biến môi trường → copy thành .env
├── Makefile                 # Lệnh setup/test/demo (bash)
├── render_demo.py           # Render 3 demo Remotion zero-key
├── requirements*.txt         # Phụ thuộc Python (core / dev / gpu / local)
├──
├── tools/                   # 167 file – "tay chân" của agent
│   ├── base_tool.py         # BaseTool / ToolContract
│   ├── tool_registry.py     # Khám phá & báo cáo năng lực
│   ├── cost_tracker.py      # Quản trị ngân sách
│   ├── video/               # video_compose, video_stitch, 20+ provider sinh video
│   ├── audio/               # piper_tts, tts_selector, audio_mixer, music_gen, ...
│   ├── graphics/            # image_selector, diagram_gen, threejs_world, ...
│   ├── analysis/            # transcriber, scene_detect, frame_sampler, ...
│   └── subtitle/            # subtitle_gen
├── pipeline_defs/           # 13 manifest YAML khai báo từng pipeline
├── skills/                  # Lớp 2 – director skill cho từng stage
│   ├── pipelines/           # explainer/, cinematic/, character-animation/, ...
│   ├── meta/                # reviewer, checkpoint-protocol, onboarding, ...
│   └── core/, creative/
├── .agents/skills/          # Lớp 3 – kiến thức công nghệ gốc (GSAP, Remotion, ...)
├── schemas/                 # JSON Schema cho artifact / checkpoint / pipeline / playbook
├── styles/                  # 5 style playbook YAML (clean-professional, ...)
├── remotion-composer/       # Engine dựng video bằng React/Remotion
├── backlot/                 # Bảng theo dõi tiến trình sản xuất (local server)
├── lib/                     # Hạ tầng: checkpoint, config, pipeline_loader, env_loader
├── scripts/                 # Script tiện ích (simulate run, scaffold, smoke test)
├── tests/                   # Contract test + QA test + eval harness
│   └── qa/                  # test_01..test_09 – chạy được, nhiều test $0
├── docs/                    # Tài liệu (bạn đang đọc)

```

2.5 Trạng thái được lưu ở đâu

Mọi thứ về một lần sản xuất nằm trong `projects/<project-id>/`:

```

projects/<project-id>/
├── project.json              # Marker: id, title, pipeline_type, style_playbook
├── checkpoint_<stage>.json  # Trạng thái từng stage (+ cost_snapshot, artifacts)
├── history/                  # Checkpoint cũ khi stage bị chạy lại (không mất lịch sử)
├── events.jsonl              # Log hoạt động (do BaseTool ghi) → Backlot đọc
├── decision_log.json         # Nhật ký quyết định (merge từ artifact decision_log)
├── artifacts/                # JSON artifact từng stage
├── assets/

```

```
├── images/ video/ audio/ music/
│   └── subtitles.srt
└── renders/final.mp4      # SẢN PHẨM CUỐI
```

Đừng nhầm thứ tự xuất hiện. `init_project()` chỉ tạo **6 thư mục con** (`artifacts/`, `assets/images|video|audio|music`, `renders/`) và `project.json` (`lib/checkpoint.py:216-248`). Các file `checkpoint_*.json`, `history/`, `events.jsonl`, `decision_log.json`, `assets/subtitles.srt` **xuất hiện dần trong quá trình chạy** — sau khi cài đặt, `projects/` thậm chí chưa tồn tại.

Thư mục `projects/` được gitignore (`.gitignore:29`) — mọi asset đều tái tạo được.

3. Yêu cầu hệ thống

Thành phần	Bắt buộc?	Phiên bản	Dùng để làm gì	Ghi chú
Python	☒ Bắt buộc	3.10+	Chạy toàn bộ tool + registry + checkpoint	Repo pin 3.10 (<code>.python-version</code>), CI dùng 3.11 (<code>.github/workflows/ci.yml:30</code>)
FFmpeg (+ ffprobe)	☒ Bắt buộc	Bản mới bất kỳ	Cắt/ghép/encode/mix/nghẹm thu video	<code>video_compose</code> khai báo <code>dependencies = ["cmd:ffmpeg"]</code> (<code>tools/video/video_compose.py:68</code>)
Node.js	☒ Cho kịch bản 1 & 3	≥ 18 (Remotion) ≥ 22 (HyperFrames)	Dựng video bằng React (Remotion) hoặc HTML/GSAP (HyperFrames)	README ghi "18+", nhưng HyperFrames cần ≥ 22 — xem 12.2
npm / npx	☒ Đi kèm Node	—	Cài & gọi Remotion/HyperFrames	
Git	Khuyến nghị	—	Clone repo	
GNU Make	☐ Tùy chọn	—	Chạy <code>make setup</code> , <code>make demo</code> , <code>make test</code>	Windows không có sẵn. Makefile dùng cú pháp bash → xem 4.1
GPU NVIDIA	☐ Tùy chọn	≥ 6 GB VRAM	Sinh video local miễn phí (Wan, Hunyuan, LTX...)	<code>requirements-gpu.txt</code>
API key	☐ Tùy chọn	—	Chất lượng cao hơn (ảnh AI, giọng cloud, nhạc)	Không cần key vẫn ra video

3.1 "Zero-key" thực tế cần những gì

Nói "không cần API key" là đúng, nhưng **không có nghĩa là không cần cài gì**. Danh sách trung thực để sản xuất video zero-key trên một máy Windows sạch:

- FFmpeg + ffprobe** — không có thì gần như toàn bộ `video_post`, `audio_processing`, `analysis` đều UNAVAILABLE.
- Python 3.10+** — để chạy registry, checkpoint, và mọi tool.
- Node ≥ 22 + npm install trong remotion-composer** — đây là chỗ dễ bị bỏ sót nhất. Không có `node_modules` thì Remotion không khả dụng, và vì `_compose` từ chối ảnh tĩnh, bạn **mất hoàn toàn** khả năng dựng video từ ảnh.
- Binary piper trên PATH** — nếu không, không có thuyết minh offline (vẫn có thể dùng TTS cloud nếu có key).

Chỉ riêng Python + `subtitle_gen` là đã chạy được (**tool duy nhất trong repo không có phụ thuộc nào** — pure Python, không FFmpeg, không key). Đó là bài kiểm tra "Python đã cài đúng chưa" rẻ nhất.

4. Cài đặt

4.1 Windows (PowerShell) — khuyến nghị

Vì sao không dùng make setup? Makefile:3-4 dùng `command -v, $(shell ...), for dir in ...; do` — đây là cú pháp bash. Trên PowerShell mặc định sẽ báo `make : The term 'make' is not recognized.` Bạn có 2 lựa chọn: chạy `make setup` trong **Git Bash / WSL**, hoặc dùng chuỗi lệnh PowerShell tương đương dưới đây (khuyến nghị, vì kiểm soát được từng bước).

```
# — Bước 0: lấy source —————
git clone https://github.com/hungnt9801/OpenMontageVideo.git
cd OpenMontageVideo

# — Bước 1: môi trường ảo Python —————
py -3.10 -m venv .venv
.\venv\Scripts\Activate.ps1
# Nếu PowerShell chặn script, chạy 1 lần cho phiên hiện tại:
# Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

# — Bước 2: thư viện Python lỗi —————
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# — Bước 3: engine dựng video Remotion —————
cd remotion-composer
npm install
```

```
cd ..

# — Bước 4: TTS offline miễn phí (Piper) —————
python -m pip install piper-tts

# — Bước 5: file cấu hình —————
Copy-Item .env.example .env
```

Nếu `npm install` báo `ERR_INVALID_ARG_TYPE` (lỗi đã biết trên Windows):

```
cd remotion-composer
npx --yes npm install
cd ..
```

Kiểm tra Piper đã sẵn sàng chưa (tool cần **lệnh** `piper`, không chỉ gói Python):

```
piper --help      # nếu "not recognized" thì binary chưa vào PATH — xem ghi chú ở mục 5
```

Làm nóng cache HyperFrames (tùy chọn — tránh chờ 30–60s ở lần render đầu):

```
npx --yes hyperframes --version
```

4.2 macOS / Linux

```
git clone https://github.com/hungnt9801/OpenMontageVideo.git
cd OpenMontageVideo
make setup      # venv + pip + npm install + piper-tts + warm cache + tạo .env
```

`make setup` thực hiện lần lượt (Makefile:54-76):

1. Tạo `.env` (ưu tiên dùng `venv`/`conda` đang active, hoặc `uv` nếu có).
2. `pip install -r requirements.txt`.
3. `cd remotion-composer && npm install`.
4. `pip install piper-tts` (nếu lỗi thì bỏ qua, dùng TTS cloud).
5. Warm cache HyperFrames qua `npx --yes hyperframes --version`.
6. Copy `.env.example` → `.env` nếu chưa có.
7. In ra `HyperFrames runtime_available=...` để bạn biết runtime đã sẵn sàng chưa.

Không có `make`:

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
(cd remotion-composer && npm install)
python -m pip install piper-tts
cp .env.example .env
```

4.3 Cài FFmpeg

Windows:

```
winget install --id Gyan.FFmpeg -e
# hoặc nếu bạn dùng Chocolatey:
# choco install ffmpeg -y
```

macOS: `brew install ffmpeg` **Ubuntu/Debian:** `sudo apt update && sudo apt install -y ffmpeg`

Mở **terminal mới** rồi kiểm tra:

```
ffmpeg -version
ffprobe -version
```

⚠️ Quan trọng: `ffprobe` phải có cùng chỗ với `ffmpeg`. Hệ thống dùng `ffprobe` để nghiệm thu video sau khi render (kiểm tra độ phân giải, fps, codec, sự tồn tại của track audio).

Đáng chú ý: `video_compose` chỉ khai báo `cmd:ffmpeg`, **không** khai báo `ffprobe` (`tools/video/video_compose.py:68`). Nếu thiếu `ffprobe`, nó **không báo lỗi** — nó lặng lẽ cho rằng video không có audio và tự ghép một track im lặng, đồng thời ghi vào bước nghiệm thu `"ffprobe not found – cannot validate output"`. Đây là kiểu hổng im lặng dễ bị bỏ qua, nên hãy kiểm tra `ffprobe -version` trước khi sản xuất thật.

4.4 Cài thêm gói "cục bộ" (`requirements-local.txt`)

Repo này có một file **bổ sung cục bộ** mà `make setup` **không** cài:

```
python -m pip install -r requirements-local.txt
```

File này (`requirements-local.txt:1-13`) giải thích rõ nó bù 2 phụ thuộc mà repo upstream thiếu:

Gói	Cần cho	Hậu quả nếu thiếu
<code>faster-whisper</code>	<code>tools/analysis/transcriber.py</code> — đường STT mặc định	Không sinh được subtitle / không soát được nội dung audio
<code>playwright</code>	<code>tools/character/character_animation.py</code> , các script <code>scripts/backlot_*.py</code> , <code>tests/backlot/test_ui_bug_bash.py</code>	Pipeline <code>character-animation</code> và các script chụp ảnh Backlot không chạy

Nếu cài `playwright`, **luôn cài kèm browser**:

```
python -m playwright install chromium
```

4.5 GPU (tùy chọn)

```
python -m pip install -r requirements-gpu.txt
python -m pip install diffusers transformers accelerate
```

Rồi thêm vào `.env`:

```
VIDEO_GEN_LOCAL_ENABLED=true
VIDEO_GEN_LOCAL_MODEL=wan2.2-ti2v-5b
```

Các model local và VRAM tương ứng (`docs/PROVIDERS.md:1264-1270`): `wan2.1-1.3b` (6 GB), `1tx2-local` (8 GB), `cogvideo-5b` (10 GB), `hunyuan-1.5` (12 GB), `wan2.2-ti2v-5b` (12 GB), `wan2.1-14b` (24 GB).

4.6 Kiểm tra sau khi cài

```
python --version           # 3.10+
ffmpeg --version           # có phiên bản
node --version             # >= 18 (>= 22 nếu dùng HyperFrames)
npm --version

# Registry nhận diện được tool?
python -c "from tools.tool_registry import registry; import json; registry.discover();
print(json.dumps(registry.provider_menu_summary(), indent=2))"
```

Lệnh cuối là **preflight** — chỉ tiết ở mục sau.

5. Cấu hình (`.env`)

`.env` là **tùy chọn, không bắt buộc**. `lib/env_loader.py:15-21` chỉ nạp file nếu nó tồn tại và **im lặng bỏ qua** nếu không có. Nghĩa là bạn có thể chạy kịch bản 1 và 2 ngay mà không cần tạo `.env`.

Cấu trúc `.env`:

```
Copy-Item .env.example .env
notepad .env
```

Đường zero-key (mặc định): để trống hết. Bạn vẫn có:

Năng lực	Tool miễn phí	Ghi chú
Thuyết minh	<code>piper_tts</code>	Offline. Cần binary piper trên PATH, không chỉ gói pip — xem cảnh báo bên dưới
Footage thật	<code>direct_clip_search</code> + các nguồn trong <code>tools/video/stock_sources/</code>	Archive.org, NASA, Wikimedia, LoC, NARA, Pond5 PD, Coverr, Mixkit, Dareful, JAXA, ESA, NOAA — không cần key
Nhạc nền	<code>music_library</code>	Quét thư mục, không phụ thuộc gì. Thả file <code>.mp3</code> vào <code>music_library/</code> (được gitignore)
Dựng video	<code>video_compose</code> , <code>video_stitch</code> , <code>video_trimmer</code> , <code>silence_cutter</code> , <code>auto_reframe</code> , <code>showcase_card</code> , <code>hyperframes_compose</code>	Cần FFmpeg / Node
Xử lý audio	<code>audio_mixer</code> , <code>audio_enhance</code>	Chỉ cần FFmpeg
Phụ đề	<code>subtitle_gen</code>	Không cần gì cả — pure Python, không FFmpeg, không key. Đây là tool duy nhất trong repo không có phụ thuộc nào
Đồ hoạ / sơ đồ	<code>diagram_gen</code> , <code>code_snippet</code> , <code>math_animate</code>	Cần <code>mmdc</code> hoặc PIL, <code>pygments</code> , <code>manim</code>
Nhân vật hoạt động	<code>character_spec_generator</code> , <code>svg_rig_builder</code> , <code>pose_library_builder</code> , <code>action_timeline_compiler</code> , <code>character_rig_renderer</code> , <code>character_animation_reviewer</code>	Không khai báo phụ thuộc nào
Phân tích	<code>scene_detect</code> , <code>frame_sampler</code> , <code>audio_probe</code> , <code>audio_energy</code> , <code>visual_qa</code> , <code>composition_validator</code> , <code>video_analyzer</code>	Chỉ cần FFmpeg/ffprobe
Đóng gói	<code>export_bundle</code>	Pure filesystem, không upload

⚠ **Piper không "pip install là xong".** `piper_tts` khai `dependencies = ["cmd:piper"]` (`tools/audio/piper_tts.py:36`) — nó cần **lệnh piper** trên PATH, việc `import` được gói Python là **không đủ** (có contract test khẳng định đúng điều này). Gói `piper-tts` khi cài sẽ tạo console script tên `piper`; nếu không, tải binary standalone từ repo `rhasspy/piper`.

⚠ **Lần đầu tổng hợp giọng cần mạng.** Bản thân tool **không tự tải model** — nó chỉ truyền `--model <tên>`. Chính CLI Piper mới tải voice model ở lần chạy đầu rồi cache lại. Muốn offline hoàn toàn, hãy tải trước voice model (ví dụ `piper --download-dir ~/.piper/models --model en_US-lessac-medium`) rồi mới ngắt mạng.

⚠ **transcriber cũng tải weight lần đầu.** `tools/analysis/transcriber.py` gọi Whisper **không** có `local_files_only`, nên lần chạy đầu sẽ tải weight từ HuggingFace rồi cache.

Các nguồn stock CẦN key (vẫn miễn phí, chỉ cần đăng ký): `pexels` (`PEXELS_API_KEY`), `unsplash` (`UNSPLASH_ACCESS_KEY`), `pixabay_video` (`PIXABAY_API_KEY`), `videvo` (`VIDEVO_API_KEY`).

Đường có key (chất lượng cao hơn): mở `.env` và điền key bạn có. Một vài nhóm đáng chú ý:

Biến	Mở khóa
<code>FAL_KEY</code>	FLUX (ảnh) + Veo/Kling/MiniMax (video) + Recraft — <i>một key mở nhiều provider</i>
<code>GOOGLE_API_KEY</code>	Imagen (ảnh) + Google TTS (700+ giọng) + Gemini Omni (video)
<code>ELEVENLABS_API_KEY</code>	TTS cao cấp + nhạc + sound effect
<code>ARK_API_KEY</code>	Seedance 2.0/2.5 trực tiếp (Volcengine Ark)
<code>PEXELS_API_KEY</code> / <code>PIXABAY_API_KEY</code> / <code>UNSPLASH_ACCESS_KEY</code>	Stock footage/ảnh miễn phí (key miễn phí)
<code>SUNO_API_KEY</code>	Sinh nhạc
<code>HEYGEN_API_KEY</code>	VEO, Sora, Runway, Kling qua một gateway

Xem toàn bộ danh sách trong `.env.example` (134 dòng, có chú thích và link lấy key cho từng nhóm).

💡 **Đừng hardcode tên key vào ghi chú của bạn.** Danh sách key hợp lệ nằm ở `dependencies = ["env:..."]` trong từng tool — xem [mục 6](#).

6. Preflight — xem bạn thực sự có gì

Trước khi sản xuất, hãy để registry tự khai báo năng lực. **Đừng tin doc cũ, kể cả tài liệu này.**

```
# Bản tóm tắt dành cho người đọc (khuyến nghị – gọn, dễ hiểu)
python -c "from tools.tool_registry import registry; import json; registry.discover();
print(json.dumps(registry.provider_menu_summary(), indent=2))"
```

Kết quả trả về 4 nhóm thông tin:

Trường	Ý nghĩa
composition_runtimes	Bật/tắt của <code>ffmpeg</code> , <code>remotion</code> , <code>hyperframes</code>
capabilities[]	Từng họ năng lực, dạng <code>đã cấu hình / tổng</code> , kèm danh sách provider
setup_offers[]	Tool chưa dùng được nhưng chỉ cần thêm 1 biến môi trường
runtime_warnings[]	Cảnh báo cụ thể, ví dụ <code>hyperframes: npm package not resolvable</code>

Đọc xong, hãy tự dịch thành câu: "Tôi đang có X/Y provider cho sinh video, có thể làm được gì ngay, và mở thêm được gì nếu bỏ 1 phút điền key."

Các lệnh đào sâu hơn (chỉ dùng khi bản tóm tắt chưa đủ):

```
# Menu đầy đủ, nhóm theo năng lực
python -c "from tools.tool_registry import registry; import json; registry.discover(); print(json.dumps(registry.provider_menu(), indent=2))"

# Catalog theo năng lực / theo nhà cung cấp
python -c "from tools.tool_registry import registry; import json; registry.discover(); print(json.dumps(registry.capability_catalog(), indent=2))"
python -c "from tools.tool_registry import registry; import json; registry.discover(); print(json.dumps(registry.provider_catalog(), indent=2))"

# Hỏi thẳng 3 engine dựng video có sẵn không
python -c "from tools.tool_registry import registry; registry.discover(); i=registry._tools['video_compose'].get_info(); print('Render engines:', i.get('render_engines')); print('HyperFrames:', i.get('hyperframes_note')); print('Remotion:', i.get('remotion_note'))"

# Kiểm tra sức khỏe HyperFrames
python -c "from tools.video.hyperframes_compose import HyperFramesCompose; r=HyperFramesCompose().execute({'operation':'doctor'}); import json; print(json.dumps(r.data, indent=2)); print('OK' if r.success else f'FAIL: {r.error}')"

```

`support_envelope()` là bản dump **khổng lồ** (có thể vài MB). Chỉ dùng khi debug — đừng dán vào chat.

6.1 Hai runtime dựng video

`video_compose` có **3 engine**, và lựa chọn được **khóa ở stage proposal** rồi mang nguyên xuống compose:

Engine	Hợp với	Cần
FFmpeg	Cắt/ghép clip video, burn phụ đề, encode	ffmpeg binary (luôn có)
Remotion	Ảnh tĩnh → video động, text card, stat card, biểu đồ, chuyển cảnh lò xo, caption từng chữ	Node + <code>remotion-composer/node_modules</code>
HyperFrames	Kinetic typography, promo sản phẩm, launch reel, website-to-video, SVG character rig	Node ≥ 22 + FFmpeg + <code>npx</code>

`edit_decisions.render_runtime` **bắt buộc** phải có, nếu thiếu `video_compose` báo lỗi và **không** tự chọn hộ (`tools/video/video_compose.py:1546-1557`). Hệ thống còn kiểm tra chéo `proposal_packet.production_plan.render_runtime` với `edit_decisions.render_runtime` để phát hiện đổi runtime ngầm (:2546-2554).

Còn một trường **bắt buộc** thứ hai ít được nhắc: với `operation="render"`, `edit_decisions.renderer_family` cũng **bắt buộc** (`tools/video/video_compose.py:1493-1498`). Nó ánh xạ sang composition Remotion qua `RENDERER_FAMILY_MAP` (:738-747):

renderer_family	Composition Remotion
explainer-data, explainer-teacher, product-reveal, screen-demo, animation-first	Explainer
cinematic-trailer, documentary-montage	CinematicRenderer
presenter	TalkingHead

Giá trị hợp lệ có thể xem bằng Python; giá trị lạ sẽ báo `Unknown renderer_family ... Valid families: [...]`. Nếu bỏ trống, code mặc định `explainer-data` (:1995) nhưng validation vẫn có thể chặn — vì vậy hãy khai báo tường minh.

⚠ operation="compose" KHÔNG nhận ảnh tĩnh. `_compose` từ chối thẳng (`tools/video/video_compose.py:523-532`): "Still image '...' in cuts. Use `operation='render'` (auto-routes to Remotion) or `operation='remotion_render'` for compositions with images, animations, or component scenes." Nghĩa là **không có đường Ken Burns bằng FFmpeg** — muốn dựng video từ ảnh thì **buộc phải có Remotion đã npm install**. Xem thêm 12.2 mục #11.

6.2 Preflight không hoàn toàn "offline và tức thì"

Chạy `provider_menu_summary()` **không cần API key**, nhưng không phải lệnh rẻ:

Việc xảy ra ngầm	Thời gian	Ghi chú
<code>npm view hyperframes version</code>	tối đa 5s	Kiểm tra gói npm có resolve được không. Offline → <code>timeout (5s)</code> – offline or slow registry

Việc xảy ra ngầm	Thời gian	Ghi chú
<code>npx --yes hyperframes doctor --json</code>	tối đa 20s	--yes có thể tải/cài gói hyperframes từ npm nếu cache chưa có
Probe ComfyUI (localhost:8188/system_stats)	tối đa 5s × 3 tool	Chỉ khi bạn dùng nhóm ComfyUI
<code>env: lookup</code>	~0s	Chỉ đọc <code>os.environ</code>

Kết quả được cache theo tiến trình. Lần đầu chạy preflight trên máy có Node nhưng mạng chậm có thể mất **5–25 giây** — hãy kiên nhẫn, đây không phải treo.

7. Kịch bản 1 — Video đầu tiên, không cần API key

Mục tiêu: có file `.mp4` trong vài phút, chi phí \$0, không cần key, không cần AI agent.

Đây là 3 demo dựng sẵn bằng component Remotion thuần (`remotion-composer/public/demo-props/*.json`):

Demo	Thời lượng	Nội dung
<code>world-in-numbers</code>	~45s	KPI grid, bar chart, pie chart, line chart, comparison card, stat reveal
<code>code-to-screen</code>	~50s	Giáo dục dev: vòng đời HTTP request với progress bar, chart, callout
<code>focusflow-pitch</code>	~40s	Pitch deck startup: chỉ số traction, donut chart doanh thu, testimonial

Cách A — có Python (khuyến nghị)

```
# Xem danh sách demo
python render_demo.py --list

# Render một demo
python render_demo.py world-in-numbers

# Render tất cả
python render_demo.py
```

Kết quả: `projects/demos/renders/<tên-demo>.mp4`

Script `render_demo.py` tự động:

- Kiểm tra `node`, `npm`, `npx` có trên PATH (`render_demo.py:49-59`).
- Chạy `npm install` trong `remotion-composer/` nếu chưa có `node_modules` (:61-63).
- Validate file props có ít nhất 1 `cut` (:68-73).
- Gọi `npx remotion render src/index.tsx Explainer <output> --props <props> --codec h264` (:87-102).

Cách B — chỉ cần Node, không cần Python

Hữu ích khi máy bạn chưa cài Python. Lệnh dưới đây là nguyên văn thao tác mà `render_demo.py` thực hiện:

```
cd remotion-composer
npm install

npx remotion render src/index.tsx Explainer `
  ..\projects\demos\renders\world-in-numbers.mp4 `
  --props public/demo-props/world-in-numbers.json `
  --codec h264
```

Muốn xem trước và chỉnh sửa trực quan (Remotion Studio):

```
cd remotion-composer
npx remotion studio
```

Lần render đầu tiên sẽ chậm

Remotion tải sẵn bản FFmpeg riêng của nó (~vài chục MB) ở lần render đầu. Đây là lý do lần đầu mất 2–5 phút, các lần sau nhanh hơn nhiều. **Điều này không cần FFmpeg hệ thống** — nhưng bạn vẫn cần FFmpeg hệ thống cho các pipeline đầy đủ (mục 8, 9).

8. Kịch bản 2 — Chạy thử toàn pipeline 8 stage (ra MP4 thật)

Mục tiêu: kiểm tra rằng Python + FFmpeg + schema + checkpoint + cost tracker + tool compose đều hoạt động, mà **không tốn một đồng nào**.

tests/qa/test_08_end_to_end.py mô phỏng đúng pipeline `animated-explainer`: nó tạo fixture bằng FFmpeg, validate artifact qua JSON Schema ở từng stage, ghi checkpoint, rồi **chạy tool thật** (`audio_mixer` + `video_compose`) để dựng ra video cuối.

```
python tests/qa/test_08_end_to_end.py
```

Đầu ra:

Đường dẫn	Nội dung
tests/qa/output/e2e_final_output.mp4	Video ~60s thật (1280x720, h264 + audio)
tests/qa/output/e2e_pipeline/qa_e2e_test/	Checkpoint của cả 8 stage
tests/qa/output/e2e_assets/	Fixture audio/video/ảnh

Script in ra `PASS/FAIL` cho từng kiểm tra, và kết thúc bằng `END-TO-END TEST COMPLETE: N passed, M failed`.

Kiểm tra riêng engine dựng video (cắt, burn phụ đề, overlay, encode profile):

```
python tests/qa/test_05_video_compose.py
python tests/qa/test_04_audio_mix.py
python tests/qa/test_06_video_stitch.py
```

tests/qa/QA_PLAN.md liệt kê đủ 8 script QA, script nào cần key và chi phí ước tính:

Script	Tool	Cần key?	Chi phí
test_04_audio_mix.py	audio_mixer	Không	\$0
test_05_video_compose.py	video_compose	Không	\$0
test_06_video_stitch.py	video_stitch	Không	\$0
test_07_playbook_intelligence.py	playbook_loader	Không	\$0
test_08_end_to_end.py	Toàn pipeline	Không	\$0
test_01_tts.py	elevenlabs_tts	ELEVENLABS_API_KEY	~\$0.02
test_02_image_gen.py	GPT Image 2 / FLUX	OPENAI_API_KEY, FAL_API_KEY	~\$0.15
test_03_music.py	music_gen	ELEVENLABS_API_KEY	~\$0.10

Chạy bộ contract test (không cần key, dùng để xác nhận bản cài đặt lành mạnh):

```
python -m pytest tests/contracts/ -q          # tương đương: make test-contracts
python -m pytest tests/ -q                    # toàn bộ
```

Nếu chưa cài `pytest`: `python -m pip install -r requirements-dev.txt`.

9. Kịch bản 3 — Tạo video thật bằng AI agent

Đây là cách OpenMontage được thiết kế để dùng. **Bạn không gõ lệnh pipeline — bạn ra đề bài, agent vận hành pipeline.**

9.1 Chuẩn bị

- 1. Cài đặt xong [mục 4](#), đã có `.env` (có thể trống key).
- 2. Mở **thư mục repo** này bằng một AI coding assistant: Claude Code, Cursor, Copilot, Windsurf hoặc Codex. Repo đã có sẵn file điều hướng cho từng loại: `CLAUDE.md`, `CODEX.md`, `CURSOR.md`, `COPILOT.md` — tất cả đều trỏ về `AGENT_GUIDE.md`.
- 3. Đảm bảo assistant đọc được `AGENT_GUIDE.md` (file này là hợp đồng vận hành bắt buộc).

9.2 Đề bài mẫu (zero-key, \$0)

Chọn một trong các prompt sau và dán vào assistant:

```
Làm cho tôi một video explainer 45 giây về vì sao bầu trời có màu xanh.
Dùng biểu đồ và chữ động, không cần ảnh – chỉ chart, stat card và typography.
```

```
Tạo video 60 giây về mức tiêu thụ cà phê trên thế giới.
Có bar chart so sánh các nước và pie chart các loại cà phê.
```

Làm video explainer 90 giây giải thích cách git rebase hoạt động.
Dùng sơ đồ động và comparison card để so rebase với merge.
Đối tượng: lập trình viên junior.

Các prompt này lấy từ `PROMPT_GALLERY.md:27-57` — đã được kiểm nghiệm cho đường zero-key (Piper TTS + stock/free media + Remotion).

9.3 Agent sẽ làm gì

Thứ tự	Stage	Agent làm	Bạn phải làm gì
0	Preflight	Quét registry, trình bày menu năng lực thật	Đọc, biết mình có gì
1	research	Nghiên cứu chủ đề, tìm dữ liệu có nguồn	—
2	proposal	Đề xuất 3–5 concept khác nhau + pipeline + tool path + dự phí + phương án runtime (Remotion vs HyperFrames) + kế hoạch nhạc	● DUYỆT
3	script	Viết kịch bản, word count khớp thời lượng, có chỉ dẫn giọng đọc	● DUYỆT
4	scene_plan	Chia cảnh, thời lượng, asset cần cho từng cảnh	● DUYỆT
5	assets	Sinh narration (TTS), ảnh, nhạc; ghi <code>asset_manifest</code>	● DUYỆT (xem filmstrip từng cảnh)
6	edit	Quyết định cắt, overlay, phụ đề, ducking nhạc	—
7	compose	Render ra <code>renders/final.mp4</code> , tự nghiệm thu bằng ffprobe + trích frame + phân tích audio	—
8	publish	Metadata SEO, chapter, gói export	● DUYỆT

● = cổng duyệt bắt buộc (`human_approval_default: true` trong `pipeline_defs/animated-explainer.yaml`).

Khi agent dừng ở cổng duyệt, việc của bạn là đọc và trả lời. Bạn có thể: duyệt, yêu cầu sửa (tối đa 3 lần/stage), hoặc huỷ.

9.4 Sản phẩm cuối

`projects/<tên-project>/renders/final.mp4`

Kèm theo toàn bộ artifact JSON có thể kiểm tra lại (`artifacts/, checkpoint_*.json`) — đây là "dấu vết kiểm toán" của lần sản xuất: chọn provider nào, model nào, tốn bao nhiêu, phương án nào bị loại và vì sao.

9.5 Chọn pipeline khác

`animated-explainer` chỉ là một trong **13 pipeline** có trong `pipeline_defs/`:

Pipeline	Hợp với	Độ ổn định
<code>animated-explainer</code>	Từ chủ đề → explainer hoàn chỉnh	production
<code>documentary-montage</code>	Montage tư liệu thật (Archive.org, NASA, Wikimedia...), dựng corpus + CLIP retrieval	beta
<code>cinematic</code>	Trailer, teaser, mood-led	production
<code>animation</code>	Motion graphics / animation-first	production
<code>character-animation</code>	Nhân vật hoạt hình rig cục bộ	beta
<code>talking-head</code>	Video người nói từ footage có sẵn	beta
<code>screen-demo</code>	Quay màn hình, walkthrough	production
<code>clip-factory</code>	Nhiều clip ngắn từ 1 nguồn dài	beta
<code>podcast-repurpose</code>	Highlight từ podcast	beta
<code>hybrid</code>	Footage gốc + visual hỗ trợ	production
<code>avatar-spokesperson</code>	Avatar dẫn chương trình / lip-sync	production
<code>localization-dub</code>	Phụ đề, lồng tiếng, bản dịch	beta
<code>framework-smoke</code>	Smoke test 2 stage tối thiểu	test

Pipeline gắn nhãn **beta** chưa được audit đầy đủ — vẫn chạy nhưng có thể gặp trục trặc. ⚠️ `documentary-montage` có manifest trên đĩa nhưng **không** xuất hiện trong bảng "Available Pipelines" của `AGENT_GUIDE.md` — xem 12.2.

10. Backlot — xem tiến trình sản xuất

Backlot là server local, **chỉ đọc**, hiển thị quá trình sản xuất: stage nào đang sáng, kịch bản dạng trang screenplay, scene plan dạng filmstrip tự đẩy lên khi asset sinh ra, quyết định, chi phí, hoạt động. Toàn bộ dữ liệu **dẫn xuất từ đĩa** (`projects/<id>/`) — Backlot không can thiệp vào pipeline.

```
python -m backlot open <project-id> # mở board của dự án (tự start server nếu cần)
python -m backlot open              # thư viện: tất cả dự án
python -m backlot serve --port 4750 # chạy server ở foreground
```

Thông số	Giá trị
Cổng mặc định	4750 (<code>backlot/__init__.py:16</code>), đổi bằng biến <code>BACKLOT_PORT</code>
Địa chỉ bind	chỉ 127.0.0.1 (<code>backlot/__main__.py:85</code>) — không lộ ra mạng ngoài
URL của dự án	<code>http://127.0.0.1:4750/p/<project-id></code>
Phụ thuộc	<code>fastapi</code> , <code>uvicorn</code> , <code>watchfiles</code> (đã có trong <code>requirements.txt</code>) + <code>Pillow</code> + binary <code>ffmpeg</code> để tạo thumbnail

Muốn thử ngay mà chưa cần sản xuất thật:

```
python scripts/backlot_simulate_run.py # chạy demo ~1 phút
python -m backlot open backlot-demo-run
```

`scripts/backlot_simulate_run.py` không phải mock rề tiền — nó chạy một sản xuất giả **qua đúng hợp đồng thật**: `init_project`, heartbeat `in_progress`, trạng thái `awaiting_human` có gate, event của tool, và artifact được ghi dần, dùng pipeline `cinematic` với project id `backlot-demo-run`. Đây là cách tốt nhất để "xem trước" Backlot trước khi bạn sản xuất thật.

Nguồn dữ liệu của từng thành phần trên board (`backlot/README.md:18-28`):

Thành phần	Đọc từ
Danh tính / thứ tự rail	<code>project.json</code> + <code>pipeline_defs/<type>.yaml</code>
Trạng thái stage, gate, version	<code>checkpoint_<stage>.json</code> + <code>history/</code>
Card kịch bản	<code>artifacts/script.json</code>
Filmstrip	<code>join scene_plan × script × asset_manifest</code>
Hiệu ứng "đang sinh", activity	<code>events.jsonl</code>
Đồng hồ chi phí	<code>checkpoint.cost_snapshot</code>
Render	<code>renders/*.mp4</code>

Có cả chế độ **REPLAY**: tua lại một lần chạy đã xong từ checkpoint history + timestamp event.

Backlot hóng thì cứ tiếp tục sản xuất. Board là bên quan sát, **không bao giờ** là blocker.

Lệnh `open` được thiết kế để suy giảm "êm": module chỉ import thư viện chuẩn nên chạy được **kể cả khi chưa cài fastapi/uvicorn**. Nếu server không khởi động hoặc không trả lời `/api/health` trong 15 giây, nó in `backlot: could not start server (...) – continuing without the board` rồi thoát với mã 1 — pipeline vẫn chạy bình thường.

Ngược lại, lệnh `serve` **không** êm như vậy: thiếu `uvicorn` sẽ ra traceback `ImportError` thẳng.

11. Xử lý sự cố

Triệu chứng	Nguyên nhân	Cách sửa
<code>python --version</code> không in gì, hoặc báo "Python was not found; run without arguments to install from the Microsoft Store"	Bạn đang gọi stub alias của Microsoft Store (<code>C:\Users\<bạn>\AppData\Local\Microsoft\WindowsApps\python.exe</code>), không phải Python thật	Cài Python từ python.org, rồi tắt alias: Settings → Apps → Advanced app settings → App execution aliases → tắt <code>python.exe</code> / <code>python3.exe</code> . Mở terminal mới
<code>make</code> : The term 'make' is not recognized	Windows không có GNU Make	Dùng lệnh PowerShell tương đương ở 4.1, hoặc chạy <code>make</code> trong Git Bash / WSL
<code>make</code> chạy nhưng lỗi cú pháp <code>command -v: not found</code>	Makefile viết cho bash, đang bị PowerShell/cmd thực thi	Dùng Git Bash hoặc WSL
<code>ffmpeg</code> : The term 'ffmpeg' is not recognized	FFmpeg chưa cài hoặc chưa vào PATH	Cài theo 4.3, mở terminal mới
<code>npm install</code> báo <code>ERR_INVALID_ARG_TYPE</code>	Lỗi đã biết trên Windows	<code>npx --yes npm install</code>
<code>Set-ExecutionPolicy</code> / "running scripts is disabled on this system"	PowerShell chặn <code>Activate.ps1</code>	<code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code>

Triệu chứng	Nguyên nhân	Cách sửa
Render đầu tiên rất chậm	Remotion đang tải bundle FFmpeg riêng	Chờ; các lần sau nhanh hơn
<code>hyperframes ... not resolvable</code> , hoặc HyperFrames báo thiếu runtime	Node < 22, thiếu <code>ffmpeg/npx</code> , hoặc cache npx chưa có	<code>node --version</code> phải ≥ 22; chạy <code>npx --yes hyperframes --version</code> ; kiểm tra sức khỏe bằng lệnh ở mục 6
Still image 'x.png' in cuts. Use operation='render' ...	Bạn gọi <code>operation="compose"</code> nhưng <code>cuts</code> có ảnh tĩnh. <code>_compose</code> cố ý từ chối ảnh	Đổi sang <code>operation="render"</code> (tự route sang Remotion), và đảm bảo <code>cd remotion-composer && npm install</code> đã chạy. Không có đường FFmpeg Ken Burns
No renderer_family in edit_decisions	Thiếu <code>renderer_family</code> — trường bắt buộc thứ hai của <code>operation="render"</code>	Thêm <code>"renderer_family": "explainer-data"</code> (hoặc <code>cinematic-trailer</code> , <code>presenter</code> , ...). Xem 6.1
Remotion project exists but node_modules are NOT installed	<code>remotion-composer/</code> có nhưng chưa <code>npm install</code>	<code>cd remotion-composer; npm install</code>
Preflight treo 5–25 giây	Bình thường — đang probe npm registry + <code>npx hyperframes doctor</code> + ComfyUI localhost	Chờ. Kết quả được cache theo tiến trình. Xem 6.2
<code>pip install piper-tts</code> thất bại, hoặc <code>piper</code> not recognized sau khi cài	Tool cần lệnh piper trên PATH, không chỉ gói Python	Kiểm tra <code>piper --help</code> ; nếu thiếu, tải binary standalone từ repo rhasspy/piper. Hoặc bỏ qua và dùng TTS cloud (<code>ELEVENLABS_API_KEY</code> , <code>GOOGLE_API_KEY</code> , <code>OPENAI_API_KEY</code>)
Transcript/subtitle trống, hoặc character-animation báo thiếu package	Chưa cài <code>requirements-local.txt</code>	<code>python -m pip install -r requirements-local.txt</code> rồi <code>python -m playwright install chromium</code>
GATE VIOLATION khi ghi checkpoint	Stage có <code>human_approval_default: true</code> bị ghi <code>completed</code> mà thiếu <code>human_approved=True</code>	Đây là hành vi đúng theo thiết kế. Chỉ ghi <code>completed</code> sau khi người duyệt đồng ý, hoặc ghi <code>awaiting_human</code>
PREREQUISITE VIOLATION: stage 'X' cannot advance; incomplete or missing: [...]	Bạn nhảy cóc stage, hoặc stage trước chưa <code>completed</code> , hoặc stage trước là gated mà chưa được duyệt	Chạy đúng thứ tự stage trong manifest. Lưu ý: cổng này không được mô tả trong <code>AGENT_GUIDE.md</code> — xem 12.2
Unknown pipeline_type 'X' — cannot resolve gate policy...	Gõ sai tên pipeline (kiểm tra <code>pipeline_defs/*.yaml</code>). Lỗi này bắn cả khi ghi <code>in_progress</code>	Đây là fail-closed có chủ đích: gõ sai tên pipeline không thể vô hiệu hoá việc kiểm soát cổng. Sửa tên cho đúng
ImportError: uvicorn khi chạy python -m backlot serve	Chưa cài phụ thuộc Backlot	<code>python -m pip install -r requirements.txt</code> . Hoặc dùng <code>python -m backlot open</code> (chạy được kể cả khi thiếu)
render_runtime is not set in edit_decisions	<code>edit_decisions</code> thiếu khoá <code>render_runtime</code> — trường này là bắt buộc ở tầng schema (<code>schemas/artifacts/edit_decisions.schema.json</code>)	Thêm <code>"render_runtime": "remotion"</code> (hoặc <code>hyperframes</code> / <code>ffmpeg</code>). Hệ thống cố ý không tự đoán
Muốn dự án ghi ra chỗ khác	—	Đặt biến <code>OPENMONTAGE_PROJECTS_DIR</code> (<code>lib/paths.py:17</code>)

12. Nhận xét khi review source code

12.1 Điểm mạnh của thiết kế

- 1. **Tách bạch tri thức khỏi code.** Logic sản xuất nằm ở YAML/Markdown — người không biết Python vẫn sửa được quy trình, chất lượng, tiêu chí review. Đây là lựa chọn kiến trúc hiếm và đúng cho một hệ thống "agent-first".
- 2. **Hợp đồng được ép bằng schema, không bằng niềm tin.** Artifact validate qua `schemas/artifacts/`, checkpoint qua `schemas/checkpoints/`, manifest qua `schemas/pipelines/`. Sai hợp đồng → fail fast.
- 3. **Fail-closed ở những chỗ quan trọng.** Ví dụ `_stage_requires_approval` (`lib/checkpoint.py:259`): `pipeline_type` được truyền nhưng **không** khớp manifest nào thì `raise`, chứ không im lặng bỏ qua gate. Gõ sai tên pipeline không thể vô hiệu hoá việc kiểm soát cổng.
- 4. **Chống đối runtime ngấm.** `video_compose` so `proposal.production_plan.render_runtime` với `edit_decisions.render_runtime` và gán cờ `runtime_swap_detected` (`tools/video/video_compose.py:2300-2303, :2546-2554`). Đây là loại lỗi rất khó phát hiện nếu chỉ nhìn kết quả cuối.
- 5. **Zero-key thực sự, không phải marketing.** `piper_tts` offline + 15 nguồn stock/archive công cộng (`tools/video/stock_sources/`) + Remotion + FFmpeg cho phép ra video hoàn chỉnh mà không cần key. `lib/env_loader.py` bỏ qua `.env` thiếu một cách lặng lẽ, nên không có "chết vì thiếu config".
- 6. **Mọi lần chạy đều resumable.** Checkpoint + `history/` + `get_next_stage()` cho phép dừng/tiếp tục mà không mất lịch sử stage cũ.
- 7. **Backlot là observer thuần.** Không có đường nào để board chặn pipeline (`backlot/README.md`), nên hòng UI không làm hỏng sản xuất.
- 8. **Ghi checkpoint an toàn với crash.** Ghi ra `.tmp` → lưu bản cũ vào `history/` → `os.replace()` (`lib/checkpoint.py:551-562`), nên không bao giờ để lại file checkpoint cụt.

9. Chi tiết "có ăn học" cho Windows. `history/` dùng `shutil.copyfile` chứ không `rename` (`lib/checkpoint.py:380`) — vì Windows từ chối rename file đang bị tiến trình khác giữ (Backlot watcher). Đây là loại chi tiết chỉ có được khi tác giả thật sự chạy thử trên Windows.
10. **Marker backfill**. Nếu người gọi `write_checkpoint()` quên truyền `pipeline_type`, hệ thống đọc lại từ `project.json` (`lib/checkpoint.py:442-455`) — nên quên tham số **không** làm mất hiệu lực của cổng duyệt.

12.2 Các điểm chưa nhất quán phát hiện khi review

#	Phát hiện	Bằng chứng	Ảnh hưởng	Đề xuất
1	<code>ATLASCLOUD_API_KEY</code> được README hướng dẫn và 3 tool yêu cầu, nhưng không có trong <code>.env.example</code>	README nêu key này; <code>tools/graphics/atlas_image.py:44</code> , <code>tools/video/atlas_video.py:60</code> , <code>tools/graphics/atlas_3d.py:61</code> đều khai <code>dependencies = ["env:ATLASCLOUD_API_KEY"]</code> ; <code>.env.example</code> không có dòng nào chứa <code>ATLAS</code>	Người dùng copy <code>.env.example</code> → không biết phải điền key nào để bật Atlas Cloud; phải đọc source mới biết	Thêm <code>ATLASCLOUD_API_KEY</code> <code>ATLAS_CLOUD_API_KEY</code> , <code>ATI</code> <code>tools/atlas_client.py:</code> <code>.env.example</code>
2	README ghi yêu cầu Node.js 18+ , nhưng HyperFrames cần Node ≥ 22	README "Prerequisites"; bảng engine trong <code>AGENT_GUIDE.md</code> ghi HyperFrames cần "Node.js ≥ 22"	Người dùng Node 18/20 làm theo README sẽ thấy HyperFrames "không giải thích được vì sao không chạy"	Ghi rõ trong README: Node 22 cho HyperFrames
3	<code>documentary-montage</code> có manifest và director skill đầy đủ nhưng thiếu trong bảng "Available Pipelines" của <code>AGENT_GUIDE.md</code> (bảng liệt kê 12, đĩa có 13)	<code>pipeline_defs/documentary-montage.yaml</code> tồn tại; <code>Get-ChildItem pipeline_defs/*.yaml = 13 file</code> ; <code>AGENT_GUIDE.md</code> bảng 12 dòng	Pipeline tốt bị "ẩn" khỏi cả người dùng lẫn agent — agent có thể không bao giờ đề xuất đường <code>documentary</code>	Thêm dòng <code>documentary-r</code> có sẵn phần mô tả ở đoạn c
4	<code>make setup</code> không cài <code>requirements-local.txt</code> , tức là mặc định thiếu <code>faster-whisper</code> (STT mặc định) và <code>playwright</code>	<code>requirements-local.txt:1-13</code> tự ghi rõ "dependency mà repo còn thiếu... <code>make setup</code> không cài"; <code>Makefile:54-76</code> chỉ cài <code>requirements.txt</code> + <code>piper-</code> <code>tts</code>	Sau <code>make setup</code> "thành công", người dùng vẫn hỏng chức năng <code>transcript</code> và <code>character-animation</code>	Gộp 2 dòng này vào <code>requir</code> thêm bước cài trong <code>make :</code>
5	<code>Makefile</code> không dùng được trên Windows mặc định, trong khi README có nhánh hướng dẫn Windows	<code>Makefile:3-4</code> dùng <code>command -v \$(shell ...) /</code> vòng <code>for</code> <code>bash</code> ; README lại đưa lệnh Windows riêng	Người dùng Windows bối rối giữa 2 luồng tài liệu	Thêm <code>make.ps1</code> hoặc script thức
6	<code>AGENT_GUIDE.md</code> và <code>PROJECT_CONTEXT.md</code> viết tay bảng pipeline, dễ lệch khỏi <code>pipeline_defs/</code>	Xem #3	Tài liệu lệch dần theo thời gian	Sinh bảng pipeline từ <code>pipel</code> (<code>tests/contracts/test_c</code> đã có sẵn nền tảng)
7	Cổng thứ hai PREREQUISITE VIOLATION không được mô tả ở bất kỳ tài liệu nào	<code>lib/checkpoint.py:284-346</code> (<code>_enforce_stage_prerequisites</code>) tồn tại và bắn lỗi khi stage trước chưa <code>completed</code> hoặc là <code>gated</code> chưa duyet; <code>AGENT_GUIDE.md</code> chỉ nói về <code>GATE VIOLATION</code>	Agent (và người viết script) đâm vào lỗi này mà không hiểu vì sao, vì tài liệu không nhắc tới	Bổ sung mục <code>PREREQUISITE</code> <code>AGENT_GUIDE.md</code> + <code>skills</code> <code>protocol.md</code>
8	<code>config.yaml</code> gần như "trơ" ở runtime: sửa nó không đổi hành vi hệ thống	<code>OpenMontageConfig.load()</code> không có call site nào ngoài test (<code>tests/contracts/test_phase0_contracts.py:263</code>); <code>CostTracker</code> nhận <code>mode</code> qua tham số constructor (<code>tools/cost_tracker.py:49,56</code>), không đọc config	Người dùng sửa <code>budget.total_usd /</code> <code>output.default_resolution</code> trong <code>config.yaml</code> rồi tưởng đã cấu hình — thực tế không có tác dụng	Nối <code>CostTracker</code> và profile <code>OpenMontageConfig.load</code> <code>config.yaml</code> rằng hiện chu
9	<code>docs/ARCHITECTURE.md</code> mô tả sai hợp đồng checkpoint	:245 nói checkpoint nằm trong <code>pipeline/</code> (thực tế <code>projects/<id>/</code>); :263 liệt kê status pending (không tồn tại trong schema, ghi vào sẽ fail validate); ví dụ JSON ở :247-260 thiếu <code>pipeline_type</code> là trường bắt buộc	Người đọc theo ví dụ sẽ tạo checkpoint không hợp lệ và bị <code>validate_checkpoint</code> từ chối	Cập nhật <code>docs/ARCHITECTU</code> <code>schemas/checkpoints/ch</code>
10	<code>skills/meta/checkpoint-</code> <code>protocol.md</code> gọi <code>get_next_stage(pipeline_dir,</code> <code>project_name)</code> — thiếu <code>pipeline_type</code>	:174,184; chữ ký thật là <code>get_next_stage(pipeline_dir, project_id,</code> <code>pipeline_type=None)</code> (<code>lib/checkpoint.py:620</code>). Khi thiếu <code>pipeline_type</code> , hàm rơi về thứ tự <code>STAGES</code> chuẩn	Trả sai stage kế tiếp trên 6+ / 13 pipeline : với <code>animated-</code> <code>explainer/animation/cinematic</code> nó trả <code>idea</code> ngay sau <code>proposal</code> (các pipeline này không có stage <code>idea</code>); với <code>screen-demo</code> trả <code>research</code> thay vì <code>real_capture</code>	Sửa 2 chỗ gọi trong skill ch
11	Đường "FFmpeg Ken Burns cho ảnh tĩnh" mà tài liệu hứa hẹn KHÔNG tồn tại trong code	<code>_compose</code> từ chối ảnh tĩnh (<code>tools/video/video_compose.py:523-532</code>); <code>_needs_remotion()</code> trả <code>True</code> cho mọi trường hợp trừ khi Remotion không có (:1375-1413). Trong khi đó <code>docs/PROVIDERS.md:1141</code> viết <i>"If Remotion is not</i>	Máy chưa <code>npm install</code> không thể dựng video từ ảnh — nhưng tài liệu khiến người dùng tưởng vẫn có đường lui. Đây là loại "silent unavailable path" nguy hiểm nhất	Sửa <code>docs/PROVIDERS.md:1</code> hoặc thật sự cài đặt đường

#	Phát hiện	Bằng chứng	Ảnh hưởng	Đề xuất
		<i>installed, compositions fall back to FFmpeg Ken Burns pan-and-zoom"</i>		
12	<code>renderer_family</code> là trường bắt buộc thứ hai của <code>operation="render"</code> nhưng không được nêu trong tài liệu vận hành	<code>_pre_compose_validation</code> chặn khi thiếu (<code>tools/video/video_compose.py:1493-1498</code> , thông báo "No <code>renderer_family</code> in <code>edit_decisions</code>"); <code>AGENT_GUIDE.md</code> chỉ nói <code>render_runtime</code> là bắt buộc	Người viết <code>edit_decisions</code> theo tài liệu sẽ bị chặn ở <code>compose</code> mà không hiểu vì sao	Ghi rõ cả hai trường bắt buộc (<code>renderer_family</code>) trong API schema <code>edit_decisions</code>

Các phát hiện trên đều **kiểm chứng được bằng file:đồng**. Với điều kiện bạn đã làm đủ **mục 4** (đặc biệt là `npm install` trong `remotion-composer`), chúng **không chặn** kịch bản 1–3 ở trên.

Riêng **#1, #4, #7** và **#11** có thể khiến bạn mất thời gian debug nếu gặp — nên biết trước. **#11** đặc biệt quan trọng: nếu bạn bỏ qua bước `npm install`, bạn sẽ mất khả năng dựng video từ ảnh, và thông báo lỗi sẽ không nói thẳng ra điều đó.

Nếu bạn định **sửa** repo này: #8 và #9 là hai chỗ dễ gây hiểu nhầm nhất cho người mới (sửa `config.yaml` mà không thấy gì thay đổi; làm theo ví dụ checkpoint trong `docs/ARCHITECTURE.md` rồi bị từ chối).

12.3 Giới hạn kiểm chứng của tài liệu này

Để bạn biết chính xác tài liệu đáng tin ở đâu:

Nội dung	Mức kiểm chứng
Yêu cầu hệ thống, luồng cài đặt, danh sách phụ thuộc	✅ Đọc trực tiếp <code>Makefile</code> , <code>requirements*.txt</code> , <code>.github/workflows/ci.yml</code> , <code>.python-version</code>
Kịch bản 1 (lệnh render demo)	✅ Lấy nguyên văn từ <code>render_demo.py:87-102</code> và <code>package.json</code> — ⚠️ chưa chạy thật trong môi trường soạn tài liệu (sandbox chặn tiến trình <code>node/npm</code>)
Kịch bản 2 (smoke test)	✅ Đọc trực tiếp <code>tests/qa/test_08_end_to_end.py</code> , <code>test_05_video_compose.py</code> , <code>QA_PLAN.md</code> — ⚠️ chưa chạy thật (máy soạn tài liệu không có Python/FFmpeg)
Hợp đồng tool (<code>video_compose</code> , <code>piper_tts...</code>)	✅ Đọc <code>input_schema</code> và code định tuyến trong <code>tools/video/video_compose.py</code>
API checkpoint	✅ Đọc <code>lib/checkpoint.py</code> — <code>init_project(project_id, *, title, pipeline_type, pipeline_dir=None, style_playbook=None); get_next_stage(pipeline_dir, project_id, pipeline_type=None)</code>
Schema artifact & checkpoint	✅ Đọc <code>schemas/artifacts/*.json</code> (20 schema) và <code>schemas/checkpoints/checkpoint.schema.json</code>
Backlot (cổng, bind address, suy giảm âm)	✅ Đọc <code>backlot/__main__.py</code> , <code>backlot/__init__.py</code> , <code>backlot/server.py</code> , <code>backlot/state.py</code>
Validate 2 manifest <code>framework-smoke / animated-explainer</code>	⚠️ Kiểm tra thủ công bằng cách đối chiếu từng key với <code>additionalProperties: false</code> + enum trong <code>pipeline_manifest.schema.json</code> (không chạy được <code>jsonschema</code>) — kết quả: cả hai hợp lệ
Các phát hiện ở 12.2	✅ Kiểm chứng bằng <code>grep</code> + đọc file, không suy đoán

Máy soạn tài liệu có: Node v24.16.0, npm 11.13.0, Git. **Máy soạn tài liệu thiếu:** Python thật (chỉ có stub Microsoft Store), FFmpeg, GNU Make.

13. Bảng tra cứu file quan trọng

Câu hỏi	Xem ở đâu
Hợp đồng vận hành cho agent	<code>AGENT_GUIDE.md</code>
Kiến trúc & quy ước	<code>PROJECT_CONTEXT.md</code> , <code>docs/ARCHITECTURE.md</code>
Cài đặt có gì	<code>Makefile</code> , <code>requirements.txt</code> , <code>.env.example</code>
Hệ thống có tool gì	<code>tools/tool_registry.py</code> → <code>provider_menu_summary()</code>
Pipeline gồm những stage nào	<code>pipeline_defs/<pipeline>.yaml</code>
Stage này phải làm gì	<code>skills/pipelines/<pipeline>/<stage>-director.md</code>
Artifact phải có field gì	<code>schemas/artifacts/</code>
Chính sách checkpoint/review	<code>skills/meta/checkpoint-protocol.md</code> , <code>skills/meta/reviewer.md</code>
Style playbook	<code>styles/*.yaml</code>
Provider nào cần key nào	<code>docs/PROVIDERS.md</code> + trường <code>install_instructions</code> trong registry
Prompt mẫu đã kiểm nghiệm	<code>PROMPT_GALLERY.md</code>
Bộ test QA chạy được	<code>tests/qa/QA_PLAN.md</code>
Theo dõi tiến trình	<code>backlot/README.md</code>

Câu hỏi	Xem ở đâu
Cấu hình ngân sách / output	config.yaml

Tài liệu này được tạo bằng cách đọc trực tiếp source code trong repo. Khi source thay đổi, hãy ưu tiên registry và [AGENT_GUIDE.md](#) hơn tài liệu này — nguyên tắc của chính OpenMontage là "đừng tin doc cũ, hãy hỏi registry".